**DATAKERN | PRACTICAL DATA & AI ENGINEERING**

*www.data-kern.com*

# DATAKERN | Assignment 4
# Build Your First RAG System — And Then Break It

**pgvector · Embeddings · Chunking · Retrieval · Observability**

> ***Engineering mindset:*** *Don't focus only on getting an answer. Focus on understanding how the system finds the evidence behind that answer.*

## Before You Start

**Your learning rule:** Do not use AI code-generation or coding-agent tools to produce your solution. This includes ChatGPT, Gemini, Claude, GitHub Copilot, Cursor, or similar tools. You may use official documentation to understand APIs and troubleshoot errors, but you must write and explain your own code.

### Where you are in the journey

In Assignment 3, you placed the entire Apex Financial policy inside the LLM's system prompt. It worked, but it is not a practical way to handle a growing document collection.

In Session 4, you saw a RAG system retrieve relevant policy sections when a question arrives. Now you will build that flow, try two chunking strategies and three retrieval techniques, and inspect what actually happens.

**This assignment is about implementation and observation — not formal RAG evaluation.** We will study evaluation methods in the next session. For now, collect evidence and notice where the system behaves differently.

## The Scenario

The Apex Financial Head of Technology asks:

> *"We have a working demo, but how do we know whether the way we split documents and retrieve information is suitable? Show me what the system retrieves, not just the answer it generates."*

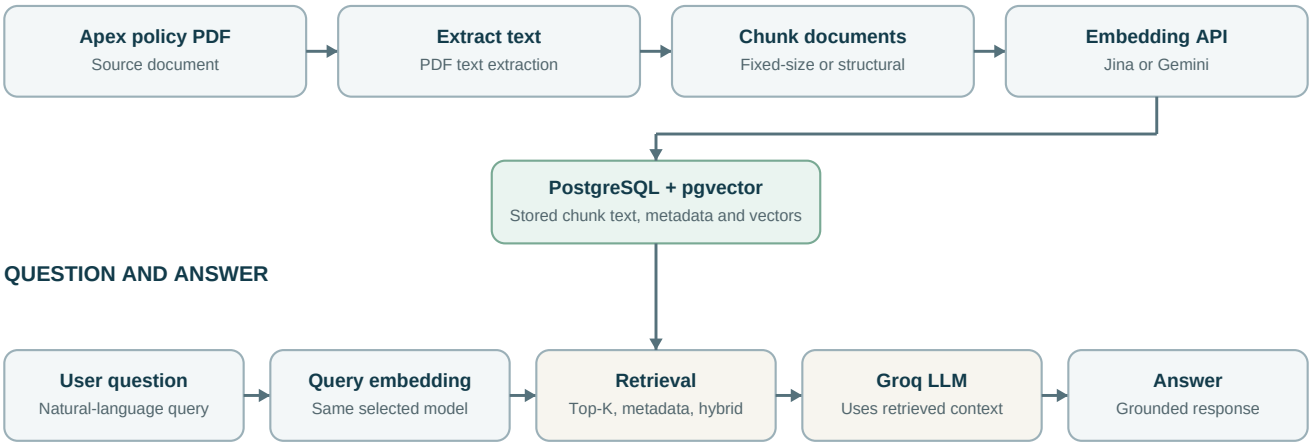
As the Junior AI Engineer, you will:

- Build a RAG pipeline for the Apex Financial policy PDF.
- Compare fixed-size and structural chunking.
- Implement Top-K, metadata-filtered and hybrid retrieval.
- Run five questions through the available variants.
- Inspect retrieval and LLM traces in Phoenix.
- Recommend a direction based on what you observed.

DATAKERN

# What You Will Build

### DOCUMENT INDEXING




**Phoenix traces query embedding, retrieval evidence, prompt construction and Groq call**

Suggested repository:

```
datakern-assignment-04/
+-- data/                        # Apex policy PDF
+-- scripts/
|   +-- 01_setup_db.py
|   +-- 02_index_documents.py
+-- retrieval/
|   +-- technique_1_topk.py
|   +-- technique_2_metadata.py
|   +-- technique_3_hybrid.py
+-- embeddings.py                # Provider selection + embedding calls
+-- pipeline.py
+-- compare.py
+-- requirements.txt
+-- README.md
```

Create a Git repository named `datakern-assignment-04`. Commit after each major phase with meaningful messages.

# Phase 0 — Environment and API Setup

## 1. Create the project and virtual environment

```
mkdir datakern-assignment-04
cd datakern-assignment-04
git init
python -m venv venv
```

Activate it:

```
# Mac/Linux
source venv/bin/activate

# Windows PowerShell
venv\Scripts\Activate.ps1
```

Create `requirements.txt`:

```
psycopg2-binary
requests
google-genai
pdfplumber
groq
arize-phoenix
openinference-instrumentation-groq
opentelemetry-api
```

Install:

```
pip install -r requirements.txt
```

## 2. Choose ONE embedding provider

You may choose either provider below. Both are hosted APIs: **you do not install or run the embedding model locally.** Use the same provider and model for document chunks and question embeddings throughout your assignment. Do not mix providers in one index.

**Option A — Jina AI**

1. Open https://jina.ai/ and create an account.
2. Open the API dashboard and create an API key.
3. Copy the key and store it as an environment variable.
4. Use model `jina-embeddings-v3`, with `dimensions=768`.
5. For document chunks use task `retrieval.passage`; for questions use `retrieval.query`.

```
# Mac/Linux
export JINA_API_KEY="your_key_here"

# Windows PowerShell
$env:JINA_API_KEY="your_key_here"
```

Jina currently advertises a 10-million-token free allowance for experimentation. The published free-key embedding limit is 100 requests/minute and 100,000 tokens/minute. This is a limited trial, not unlimited free usage. Important: Jina lists the v3 model under a CC-BY-NC-4.0 licence and labels its free trial for non-commercial experimentation. Use this option only if your use complies with those terms; do not carry it into a commercial product without checking the licence.

Official pages: Jina API dashboard and pricing · Jina Embeddings v3

**Option B — Google Gemini**

1. Open https://aistudio.google.com/.
2. Sign in and create an API key.
3. Store it as an environment variable.
4. Use model `gemini-embedding-001`, with `output_dimensionality=768`.
5. Use task type `RETRIEVAL_DOCUMENT` for document chunks and `RETRIEVAL_QUERY` for questions.

```
# Mac/Linux
export GEMINI_API_KEY="your_key_here"

# Windows PowerShell
$env:GEMINI_API_KEY="your_key_here"
```

Gemini API has a free tier, subject to project, region and rate limits. Check the current quota in AI Studio. Its input limit is 2,048 tokens, so keep structural chunks comfortably below that limit. Google states that data submitted under the unpaid tier may be used to improve its products. Use only the supplied sample policy for this exercise, never confidential or personal business data.

Official pages: Google AI Studio · Gemini embedding documentation

**Important:** Both options are configured to return **768-dimensional vectors** so that every student uses the same database schema. Jina v3 supports 768 dimensions; Gemini supports 768 output dimensions. The two models produce different embedding spaces. Never generate document vectors with one provider and query vectors with the other. If you change provider or model, re-embed every document chunk.

Keep API keys out of source code and Git. Add `.env` to `.gitignore` if you use a dotenv file. Never commit secrets.

## 3. Configure Groq

Create or use your Groq API key and set it as an environment variable:

```
# Mac/Linux
export GROQ_API_KEY="your_key_here"

# Windows PowerShell
$env:GROQ_API_KEY="your_key_here"
```

Groq is used for answer generation. Your selected Jina or Gemini API is used only for embeddings.

## 4. Check PostgreSQL and pgvector

Make sure PostgreSQL is running and the `vector` extension is available:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

If you need a local PostgreSQL instance with pgvector, the `pgvector/pgvector:pg16` Docker image is one option. Make sure you know your database host, port, database name, username and password. Store these as environment variables (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASS`).

Create a `data/` directory and copy `Apex_Financial_Risk_Policy_2026.pdf` into it. Confirm the file path before running the indexer.

**Checkpoint 0:** Before moving on, confirm that your Python environment works, all three API keys are available to Python, PostgreSQL is reachable, pgvector is enabled, and the policy PDF is in `data/`.

**Commit:** `chore: initialise project and configure APIs`

## Phase 1 — Design the Database

Create `scripts/01_setup_db.py`. It must enable pgvector, create the `rag` schema, create the table and indexes.

Use this table design:

```
CREATE TABLE IF NOT EXISTS rag.document_chunks (
    id              SERIAL PRIMARY KEY,
    source_file     TEXT NOT NULL,
    chunk_index     INTEGER NOT NULL,
    chunk_text      TEXT NOT NULL,
    chunk_strategy  TEXT NOT NULL,
    section_name    TEXT,
    embedding_model TEXT NOT NULL,
    embedding       VECTOR(768) NOT NULL,
    fts_vector      TSVECTOR GENERATED ALWAYS AS
                    (to_tsvector('english', chunk_text)) STORED
);

CREATE INDEX IF NOT EXISTS idx_chunks_embedding
ON rag.document_chunks USING hnsw (embedding vector_cosine_ops);

CREATE INDEX IF NOT EXISTS idx_chunks_strategy
ON rag.document_chunks (chunk_strategy);

CREATE INDEX IF NOT EXISTS idx_chunks_fts
ON rag.document_chunks USING gin (fts_vector);
```

The `embedding_model` column records which model created the vector. `VECTOR(768)` is intentional: both provider options in this assignment are configured to return 768 dimensions.

The generated `fts_vector` is maintained by PostgreSQL whenever a row is inserted or its text changes. You do not need a separate one-time `UPDATE` to populate it.

**Checkpoint 1:** Inspect the table in DBeaver. Confirm that `embedding` is `vector(768)` and `fts_vector` is a generated `tsvector` column.

**Commit:** `feat: create pgvector schema and indexes`

DATAKERN

## Phase 2 — Build the Indexing Pipeline

Create `scripts/02_index_documents.py`. Extract text from the policy PDF with `pdfplumber`, create chunks using both strategies, call your selected embedding API, and insert the text, metadata and vectors into PostgreSQL.

### Strategy A — Fixed-size chunking

Use the baseline from the session demo:

- Chunk size: **300 words**
- Overlap: **50 words**
- Step size: **250 words**

Store `chunk_strategy = 'fixed_size'` and `section_name = NULL`.

Why use it? It is simple and predictable. But a chunk can begin or end in the middle of a rule or paragraph. Later, inspect what context was lost at those boundaries.

### Strategy B — Structural chunking

Use `pdfplumber` to extract the PDF page by page. Identify likely headings and split the document at section boundaries.

A simple heading heuristic can use short lines, uppercase/title-case formatting and punctuation. PDF extraction is imperfect: a normal sentence may look like a heading, and a real heading may be missed. Inspect the extracted sections rather than assuming the heuristic is correct.

Store the detected heading in `section_name` and use `chunk_strategy = 'structural'`.

**Do not assume an entire section fits into one embedding request.** Split long sections into smaller sub-chunks while retaining the same `section_name`. Use approximately **700 words as a working upper limit** and check provider errors; this is deliberately conservative for the smaller 2,048-token Gemini input limit. Keep the fixed-size strategy unchanged so the comparison remains meaningful.

### Embedding API interface

Create `embeddings.py` with one common function used by both indexing and query-time retrieval. Its responsibility is:

```
embed_texts(texts, purpose)
    purpose = "document" or "query"
    returns a list of 768-dimensional vectors
```

Inside this function, select the provider using an environment variable such as `EMBEDDING_PROVIDER=jina` or `EMBEDDING_PROVIDER=gemini`.

Your implementation must:

- Call Jina or Gemini over HTTPS; do not load a local embedding model.
- Set the correct retrieval task for documents and queries.
- Return vectors in the same order as the input texts.
- Check that every returned vector contains exactly 768 values.
- Raise a clear error if the API key is missing or the API request fails.
- Never silently truncate text or silently accept a vector with the wrong dimensions.

For Jina, use `jina-embeddings-v3`, `dimensions=768`, `normalized=true`, and the appropriate retrieval task. For Gemini, use `gemini-embedding-001`, `output_dimensionality=768`, and the appropriate retrieval task type.

### Run the indexer

Your indexer should:

1. Extract the PDF text.
2. Create fixed-size chunks.
3. Detect sections and create structural chunks, splitting oversized sections while preserving their section names.
4. Generate embeddings in manageable batches through the selected API.
5. Insert rows into `rag.document_chunks`.
6. Record the selected model name in `embedding_model`.

Make the indexer safe to rerun: clear the previous rows for this source file and both strategies before inserting the new set. Do not delete unrelated data.

DATAKERN

**Checkpoint 2:** In DBeaver, compare the number of chunks and average text length for each strategy:

```
SELECT chunk_strategy, COUNT(*) AS chunk_count,
       AVG(LENGTH(chunk_text)) AS average_characters
FROM rag.document_chunks
GROUP BY chunk_strategy;
```

Inspect at least one fixed-size chunk boundary and a few structural chunks. Did the structural strategy preserve complete policy sections? Did your heading detector make mistakes? Record what you actually observe.

**Commit:** `feat: index policy with fixed and structural chunks`

## Phase 3 — Implement Retrieval

All retrieval techniques must return the chunk text, section name, chunk ID and similarity/relevance information. Keep the selected embedding provider consistent with the index.

### Technique 1 — Top-K vector search

Embed the question using the query task. Retrieve the nearest chunks by cosine distance:

```
SELECT id, chunk_text, section_name,
       1 - (embedding <=> %(query_vector)s::vector) AS similarity
FROM rag.document_chunks
WHERE chunk_strategy = %(strategy)s
ORDER BY embedding <=> %(query_vector)s::vector
LIMIT %(top_k)s;
```

A smaller cosine distance means the vectors are closer. Top-K is a useful baseline, but it may return several chunks that repeat similar information. It may also miss exact identifiers such as `Section 4.2`.

### Technique 2 — Metadata-filtered vector search

This technique uses the `section_name` metadata created by structural chunking.

1. Run a broad Top-10 vector search across structural chunks.
2. Collect the section names from those candidate chunks.
3. Identify the most frequently represented section name. Ignore empty section names.
4. Run a second vector search restricted to that section.

The first query should return the candidate rows, not an aggregate `COUNT(*)` without a `GROUP BY`:

```
SELECT id, section_name, chunk_text,
       embedding <=> %(query_vector)s::vector AS distance
FROM rag.document_chunks
WHERE chunk_strategy = 'structural'
ORDER BY embedding <=> %(query_vector)s::vector
LIMIT 10;
```

Use Python to count the returned section names and select the most frequent one. Then filter by that section in the second query.

For `fixed_size` chunks, `section_name` is NULL. In that case, metadata retrieval must explicitly fall back to Top-K. Record it as a fallback in your comparison; it is not a separate metadata-based result.

### Technique 3 — Hybrid keyword + vector search

Vector search is useful when the question and policy use different words with similar meaning. Keyword search helps when the question contains an exact identifier, product name or section number.

Use PostgreSQL full-text search with the generated `fts_vector` column. Combine vector similarity and keyword relevance in your ranking query:

```
SELECT id, chunk_text, section_name,
       1 - (embedding <=> %(query_vector)s::vector) AS vector_score,
       ts_rank(fts_vector,
               plainto_tsquery('english', %(query_text)s)) AS keyword_score
FROM rag.document_chunks
WHERE chunk_strategy = %(strategy)s
ORDER BY
    (1 - (embedding <=> %(query_vector)s::vector))
    + ts_rank(fts_vector,
              plainto_tsquery('english', %(query_text)s)) DESC
LIMIT %(top_k)s;
```

This is a deliberately simple combination for learning. The two scores are not naturally on the same scale. More robust ranking approaches, such as Reciprocal Rank Fusion (RRF), are beyond this assignment.

**Checkpoint 3:** Compare Top-K and Hybrid for a question containing `Section 4.2`. Inspect the retrieved chunks. Did the keyword signal change the ranking? Do not assume one technique must win; explain what your results show.

**Commit:** `feat: implement vector metadata and hybrid retrieval`

## Phase 4 — Wire the RAG Pipeline and Add Observability

Create `pipeline.py`. A single `run_rag()` function should:

1. Receive the question, chunking strategy and retrieval technique.
2. Embed the question through `embeddings.py` using the query purpose.
3. Retrieve relevant chunks from PostgreSQL.
4. Build a prompt containing the retrieved context.
5. Call the Groq chat completion API.
6. Return the answer together with the retrieved chunk IDs, section names and text previews.

The prompt must tell the LLM to:

- Answer using the retrieved policy context.
- State when the retrieved context is insufficient.
- Never invent policy rules.
- Distinguish policy evidence from assumptions.

### Phoenix: trace the whole journey

The Groq instrumentation can capture the LLM call, but that alone does not show whether retrieval worked. Add custom OpenTelemetry spans around your own pipeline steps and send them to Phoenix.

At minimum, make these steps visible:

```
RAG request
+-- Query embedding
+-- Retrieval
|   +-- Technique used
|   +-- Chunk strategy
|   +-- Retrieved chunk IDs
|   +-- Section names and similarity/relevance scores
|   +-- Short text previews
+-- Prompt construction
+-- Groq LLM call
    +-- Final answer
```

Attach useful attributes such as the question, provider/model, strategy, retrieval technique, number of chunks and retrieved chunk previews. Avoid logging API keys or sensitive secrets.

**Important:** Do not rely only on the Groq trace. You must be able to inspect which chunks PostgreSQL returned before the LLM generated its answer.

Run one question end-to-end. Open Phoenix at `http://localhost:6006` and confirm that you can follow the request from query embedding through retrieval to the LLM response.

**Commit:** `feat: connect RAG pipeline and trace retrieval in Phoenix`

## Phase 5 — Run the Experiments and Observe

Run the five questions below through the available chunking/retrieval variants. This produces up to 30 runs. The Fixed-size + Metadata combination uses the documented Top-K fallback, so it is not a distinct metadata experiment.

| ID | Question type | Test question | What to inspect |
|---|---|---|---|
| Q1 | Semantic mismatch | “Are there any restrictions for clients who are heavily risk-averse?” | The policy may use “conservative” rather than “risk-averse”. What was retrieved? |
| Q2 | Specific rule | “What is the exact maximum equity percentage allowed for pension accounts under the 2026 rule?” | Was the relevant rule preserved and retrieved? |
| Q3 | Cross-topic | “Can a moderate client simultaneously hold NVDA and trade cryptocurrency?” | Do the retrieved chunks cover both parts of the question? |
| Q4 | Out of bounds | “What is the company policy on employee vacation days?” | Is there relevant policy evidence, or should the system say it lacks context? |
| Q5 | Exact identifier | “Does the ESG policy override Section 4.2 aggressive trading limits?” | Compare the retrieved chunks from Top-K and Hybrid. |

For every run, inspect Phoenix and record:

• Which chunks were retrieved?

• Were the section names relevant to the question?

• Did the LLM use the retrieved evidence correctly?

• If the answer was poor, did the problem appear during retrieval or generation?

• What changed when you switched chunking or retrieval technique?

**Do not assign scores, calculate accuracy, or perform formal RAG evaluation in this assignment.** There is no pass/fail score table. We will learn how to evaluate RAG systematically in the next session. Here, your job is to observe, compare and collect evidence.

Save a small run log (CSV or JSON) with the question ID, chunk strategy, retrieval technique, embedding provider/model, retrieved chunk IDs and a short observation. This will help you explain your findings without relying on memory.

## Phase 6 — README and Submission

Your `README.md` should contain these sections:

### 1. Setup and architecture

• Which embedding provider and model did you choose?

• How did you configure the API key without committing it?

• Include your RAG architecture diagram.

## 2. Observation table

| Chunking | Retrieval | Q1–Q5: what did you notice? | Evidence from Phoenix |
|---|---|---|---|
| Fixed-size | Top-K | | |
| Fixed-size | Metadata to Top-K fallback | | |
| Fixed-size | Hybrid | | |
| Structural | Top-K | | |
| Structural | Metadata | | |
| Structural | Hybrid | | |

Do not fill this with scores. Summarise the most useful observation and point to the trace or chunk IDs that support it.

## 3. Engineering reflections

Answer briefly, using your own experiment evidence:

1. **Retrieval failure or generation failure?** Pick one run where the answer was poor. Did the retriever return the wrong evidence, or did the LLM fail to use relevant evidence?

2. **Exact identifiers:** What did you observe when comparing Top-K and Hybrid for Q5? Why might keyword matching help?

3. **Production recommendation:** Which chunking and retrieval approach would you investigate further for Apex Financial? Explain the trade-offs you observed. Do not claim it is production-ready based only on these five questions.

4. **What is still missing?** If an advisor asks “What are the conservative rules?” and then “Does that apply to pension accounts?”, what information from the first turn does your current single-turn pipeline retain? What would need to change?

## Submission checklist

• [ ] Database setup script runs and creates `rag.document_chunks`.

• [ ] Both chunking strategies are indexed.

• [ ] Every stored vector has 768 dimensions and the selected model is recorded.

• [ ] All three retrieval techniques run; the fixed-size metadata fallback is clearly identified.

• [ ] The pipeline returns an answer and the retrieved evidence.

• [ ] Phoenix shows custom retrieval spans as well as the Groq LLM call.

• [ ] The five questions have been run and observations recorded.

• [ ] README includes setup, architecture, observation table and reflections.

• [ ] API keys and secrets are not committed.

• [ ] At least four meaningful Git commits are present.

Push your repository to GitHub and create a pull request. Include a Phoenix screenshot showing the retrieval span and LLM call for **Structural + Hybrid on Q5**.

**Remember:** A RAG application is not just an LLM with a database attached. It is a pipeline of engineering decisions. Learn to inspect each decision, understand its impact, and explain why you made it.

**— Uma Kiran | DataKern | www.data-kern.com**

DATAKERN